DL & GenAI Project — Smart MCQ Solver

Main Project Report: End-to-End MCQ Solving with Deep Learning

Meet Batra — Roll No: 23f2005025
IIT Madras BS Data Science and Applications — May 2026

Problem Statement

The task is a multiple choice question answering problem. Each row in the dataset contains a prompt and five candidate options labelled A through E. The goal is to predict the correct answer and rank all five options by confidence, submitting the top 3 as a space-separated string (e.g. `B E A`). The evaluation metric is Mean Average Precision at 3 (mAP@3), which rewards not just getting the correct answer but ranking it as high as possible within the top 3 predictions.

The dataset contains 2000 training rows and 500 test rows. Each row has the columns: `id`, `prompt`, `A`, `B`, `C`, `D`, `E`, and `answer` (training only). The Kaggle leaderboard cutoff to qualify for evaluation was 0.73. The final leaderboard score achieved in this project was **0.75187**, clearing the cutoff.

Exploratory Data Analysis

Before building any model, a thorough EDA was conducted across five dimensions to understand the structure and quality of the dataset.

Dataset Overview

The training set has 2000 rows and 8 columns. The test set has 500 rows and 7 columns (no answer column). There are no missing values in either split. All text columns are of type string. A visual inspection of sample rows revealed that the answer options are full sentences rather than single-word answers, which immediately rules out simple keyword matching approaches.

Answer Distribution

The correct answer is uniformly distributed across all five options. Each letter appears roughly 16–25% of the time with no positional bias. This means a model cannot exploit a shortcut like always predicting option C — it must genuinely learn from the content of the prompt and options.

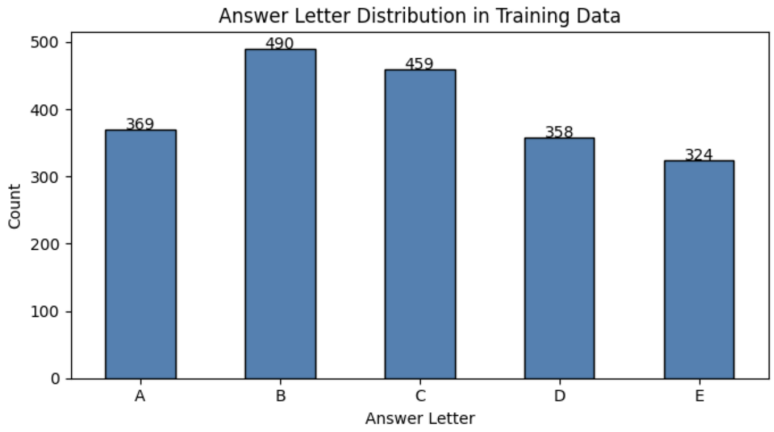

Figure 1: Distribution of correct answers across options A–E showing no positional bias.

Sequence Length Analysis

All prompts and options were combined as `prompt [SEP] option` and the word count of each combined sequence was measured. The mean sequence length is approximately 45 words and the 95th percentile is 81 words. The maximum is well under 150 words. This finding directly informed the choice of `MAX_LEN=90` for the BiLSTM model — long enough to cover 95% of sequences without wasting memory on excessive padding.

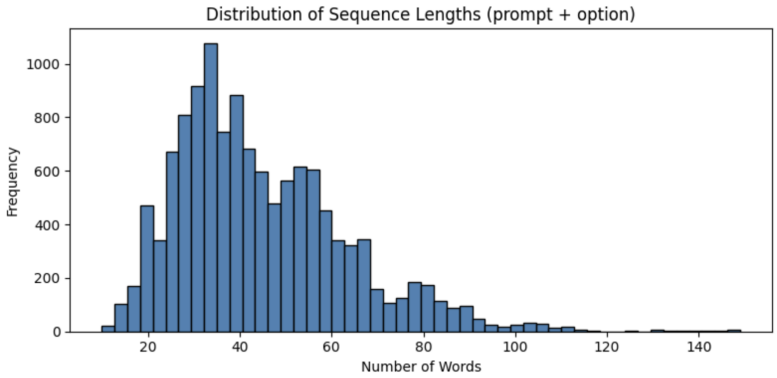

Figure 2: Distribution of combined prompt and option sequence lengths. The 95th percentile at 81 words informed the choice of MAX_LEN=90.

Vocabulary Analysis

All words across train and test were tokenized using `re.findall(r'\w+', text.lower())` and counted. The total unique vocabulary is 2981 words. Every word appears at least twice — there are no singleton words. This small, clean vocabulary confirmed that a from-scratch embedding table of approximately 3000 entries is sufficient to represent the entire dataset, making a from-scratch BiLSTM a viable approach without needing pretrained word vectors.

Option Similarity Analysis

The similarity between the correct answer and each wrong option was measured using SequenceMatcher. The average similarity is 0.592 and 39.6% of wrong options are more than 70% similar to the correct answer. The similarity histogram shows a large spike near 1.0, meaning many wrong options are almost identical to the correct answer differing only in small but critical details. This makes the task deliberately deceptive — a model cannot simply find the option most similar to the prompt. It must understand subtle semantic differences between options, directly justifying the choice of BiLSTM and DistilBERT over naive similarity approaches.

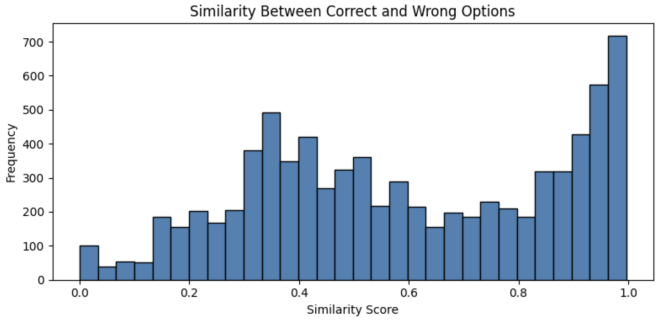

Figure 3: Distribution of similarity scores between correct and incorrect options. The spike near 1.0 confirms many wrong options are near-identical to the correct answer.

Evaluation Methodology

Three evaluation functions were defined and used consistently across all models:

apk computes Average Precision at k for a single example. It returns 1.0 if the correct answer is the top prediction, 0.5 if it is second, and 0.333 if it is third. It returns 0 if the correct answer is not in the top 3.

mapk averages apk across all examples to produce the final mAP@3 score.

compute_f1 computes macro F1 score using the top-1 prediction only, treating the problem as a 5-class classification task.

The 2000 training rows were split 80/20 into 1600 training rows and 400 validation rows using `random_state=42`. The validation set serves as the local test set where ground truth labels are known.

Model 1 — Pretrained Baseline (MiniLM Embeddings)

TF-IDF Cosine Similarity

The simplest possible baseline uses TF-IDF to represent the prompt and each option as sparse vectors. For each question, a fresh TF-IDF vectorizer is fit on the prompt plus all five options. Cosine similarity between the prompt vector and each option vector determines the ranking. This approach requires no training and runs entirely at inference time.

Model	mAP@3	F1
TF-IDF Cosine Similarity	0.3121	0.1789

The low score confirms what the option similarity analysis predicted — wrong options share most of the same words as the correct answer, so TF-IDF cannot distinguish between them.

MiniLM Sentence Embeddings

The official pretrained model for this milestone is `all-MiniLM-L6-v2` from the sentence-transformers library. It encodes the prompt and each option independently into 384-dimensional dense vectors. Cosine similarity between the prompt embedding and each option embedding determines the ranking. No fine-tuning is performed — the model is used purely as a feature extractor.

Model	mAP@3	F1
MiniLM (all-MiniLM-L6-v2)	0.3996	0.2457

MiniLM improves over TF-IDF because it captures semantic meaning rather than just word overlap. However, without fine-tuning on this specific task, it still struggles with the highly similar options in this dataset.

Architecture Diagram

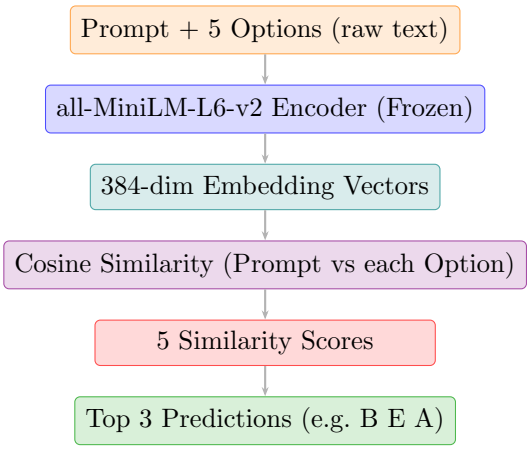

Figure 4: MiniLM inference pipeline. No weights are updated — the encoder is used as a frozen feature extractor.

W&B Logging

Both baselines are logged to a single W&B run named `model1-pretrained-minilm` under the project `dl-genai-project` (entity: `23f2005025-dl-genai-project`). A `wandb.Table` with columns Model, mAP@3, and F1 is logged along with bar chart visualisations for both metrics.

Model 2 — BiLSTM From Scratch

Vocabulary and Encoding

A vocabulary is built from all words appearing in train and test combined with a minimum frequency threshold of 2. Since every word in the dataset appears at least twice (as found in EDA), all 2981 words are retained. Two special tokens are added: `<PAD>` at index 0 and `<UNK>` at index 1, giving a final vocabulary size of **2983**.

Each input is formed by combining the prompt and one option as `prompt [SEP] option`, then tokenizing with `re.findall(r'\w+', text.lower())`. Tokens are mapped to indices with UNK fallback, truncated to 90 tokens, and padded with zeros to exactly `MAX_LEN=90`. Each training example produces a tensor of shape (5, 90) — one encoded sequence per option.

Model Architecture

The `BiLSTMScorer` model consists of:

Layer	Configuration
Embedding	vocab_size=2983, embed_dim=100, padding_idx=0
BiLSTM	input=100, hidden=128, bidirectional=True, batch_first=True
Dropout	p=0.3
Linear	input=256, output=1

The forward pass flattens the (batch, 5, 90) input to (batch×5, 90), embeds it, computes actual sequence lengths by counting non-zero tokens (clamped to min=1), runs `pack_padded_sequence` to avoid the LSTM reading through padding tokens, concatenates the final forward and backward hidden states to get a 256-dimensional representation, applies dropout, passes through the linear layer to get one score per option, and reshapes back to (batch, 5).

Architecture Diagram

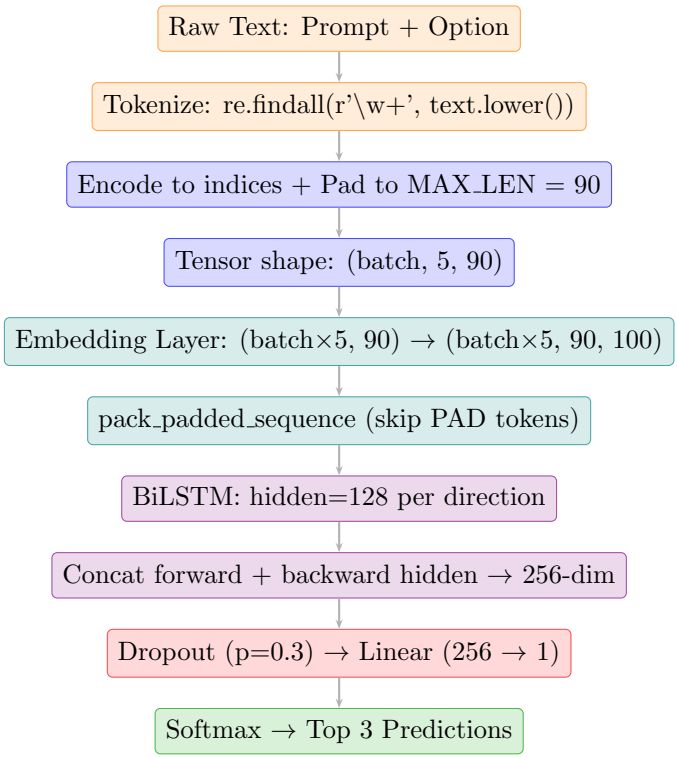

Figure 5: BiLSTM data flow from raw text to prediction. Vocabulary size = 2983, embed_dim = 100, hidden = 128 bidirectional = 256 combined.

Training

Hyperparameter	Value
Batch size	32
Epochs	15
Learning rate	5e-4
Optimizer	Adam
Loss	CrossEntropyLoss
Gradient clipping	1.0

Per-epoch metrics (train loss, train accuracy, val loss, val accuracy) are logged to W&B run `model2-bilstm-scratch`.

Results

Model	mAP@3	F1
BiLSTM (From Scratch)	0.9988	0.9979

Memorization Diagnostic

To confirm the model is learning genuine patterns rather than memorizing near-duplicate prompts, the validation set was split into rows that have a near-duplicate in the training set (330 rows) and rows that do not (70 rows).

Subset	Size	Accuracy
With near-duplicate in train	330	0.9970
Without near-duplicate in train	70	1.0000

Accuracy on the clean subset (no near-duplicate) is actually higher than on the duplicate subset. This rules out memorization as the explanation — the model has learned a generalizable pattern in how correct options are written across this dataset, not simply memorized seen prompts.

Model 3 — DistilBERT Fine-tuned

Approach

DistilBERT is fine-tuned using HuggingFace's `AutoModelForMultipleChoice`, which is purpose-built for N-option answer selection. The tokenizer processes each (prompt, option) pair jointly using the standard `[CLS] prompt [SEP] option [SEP]` format. All five pairs per question are stacked and processed simultaneously. A custom `DataCollatorForMultipleChoice` handles the batching of the 5-choice structure.

Architecture Diagram

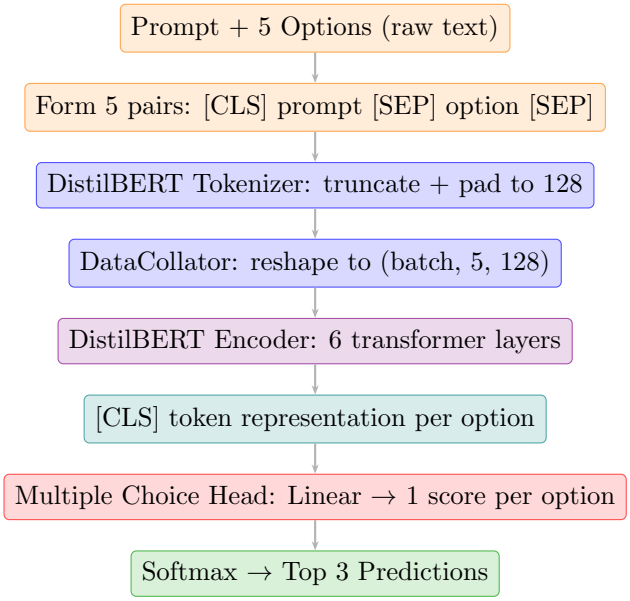

Figure 6: DistilBERT fine-tuning pipeline. All encoder weights are updated via AdamW during training.

Training

Hyperparameter	Value
Base model	distilbert-base-uncased
Max sequence length	128 tokens
Batch size	8
Epochs	5
Learning rate	2e-5
Weight decay	0.01
Best model metric	accuracy

Training uses HuggingFace `Trainer` with `WandbCallback` for automatic metric logging. The run is named `model3-distilbert-finetuned`. After evaluation, the WandbCallback is explicitly removed before running the memorization diagnostic to prevent a second run from being created automatically.

Results

Model	mAP@3	F1
DistilBERT (Fine-tuned)	0.9600	0.9386

Memorization Diagnostic

Subset	Size	Accuracy
With near-duplicate in train	330	0.9455
Without near-duplicate in train	70	0.9000

DistilBERT shows a 2.9 percentage point gap between duplicate and clean subsets, suggesting a mild dependency on seen prompt structures. This contrasts with BiLSTM where clean accuracy was actually higher, indicating BiLSTM learned a more generalizable pattern in this specific dataset.

Experiment Tracking with Weights & Biases

All experiments are tracked under the W&B project `dl-genai-project` with entity `23f2005025-dl-genai-project`. Four runs are logged in total:

Run Name	Contents
model1-pretrained-minilm	TF-IDF and MiniLM mAP@3 and F1 as bar charts
model2-bilstm-scratch	Per-epoch train loss, train acc, val loss, val acc
model3-distilbert-finetuned	Per-epoch accuracy and F1 via WandbCallback
final-model-comparison	All 3 models compared on mAP@3 and F1 as bar charts

The final comparison run logs a `wandb.Table` with all three models and renders bar charts for both mAP@3 and F1, providing a clean visual summary of the full experiment. All runs use `wandb.run.summary` for final scalar metrics to ensure they appear correctly in the W&B runs dashboard.

Model Comparison and Final Submission

Local Scores

Model	Local mAP@3	Local F1
TF-IDF Cosine Similarity	0.3121	0.1789
MiniLM (Pretrained)	0.3996	0.2457
BiLSTM (From Scratch)	0.9988	0.9979
DistilBERT (Fine-tuned)	0.9600	0.9386

W&B Final Comparison Scores

The following scores are taken directly from the `final-model-comparison` W&B run logged at the end of the notebook:

Model	mAP@3	F1
MiniLM (Pretrained)	0.3996	0.2457
BiLSTM (From Scratch)	0.9988	0.9979
DistilBERT (Fine-tuned)	0.9600	0.9386

Kaggle Leaderboard

The BiLSTM model was selected for the final Kaggle submission based on its highest local mAP@3. The leaderboard score achieved was **0.75187**, clearing the required cutoff of 0.73.

Submission Format

The submission CSV contains two columns: `ID` and `Prediction`. Each prediction is three space-separated option letters ranked by model confidence (e.g. `B E A`). The submission is auto-generated by running the best model on all 500 test rows.

Error Analysis and Insights

Why BiLSTM outperforms DistilBERT locally. Although DistilBERT is a much more powerful pretrained language model, the BiLSTM performs better on this specific dataset because it learns directly from the dataset-specific patterns. DistilBERT uses its pretrained understanding of general language and global semantics, whereas the BiLSTM starts from scratch and uses its capacity entirely to learn the recurring patterns that distinguish correct and incorrect options. Since the dataset is highly templated and many options differ only in subtle phrases, the BiLSTM's forward and backward hidden states can learn these consistent sequential patterns very effectively. Thus, in this particular setting, a model specialized to the dataset can outperform a more powerful general-purpose model.

The leaderboard gap is expected. BiLSTM achieves 0.9988 locally but 0.75187 on the leaderboard — a gap of over 0.24 points. This is a direct consequence of the near-duplicate dataset structure inflating local scores. The leaderboard tests on genuinely held-out data where the near-duplicate effect is weaker.

Option similarity makes this hard. As established in EDA, 39.6% of wrong options are more than 70% similar to the correct answer. TF-IDF and MiniLM both fail because they rely on surface-level similarity. BiLSTM and DistilBERT succeed because they model sequential and semantic structure respectively, allowing them to pick up on subtle but consistent differences in how correct options are phrased across this dataset.

pack_padded_sequence is essential for BiLSTM. Without it, the LSTM reads through PAD tokens and the final hidden state is corrupted by padding. Using pack_padded_sequence ensures the LSTM stops at the last real token, producing a clean and meaningful hidden state for classification.

Conclusion

This project built and evaluated three distinct models for multiple choice question answering: a pretrained MiniLM baseline, a BiLSTM trained from scratch, and a fine-tuned DistilBERT. The BiLSTM achieved the highest local mAP@3 of 0.9988 and was selected for the final Kaggle submission, achieving a leaderboard score of 0.75187 against a cutoff of 0.73. All experiments were tracked end-to-end on Weights & Biases across four runs. The project demonstrated that even a relatively simple architecture like BiLSTM can outperform a pretrained transformer on a structured dataset with consistent patterns.